Fachhochschule
Dortmund
University of Applied Sciences and Arts

# Homework Assignment Report

## Subject: MicroElectronics

## Supervisor: Dr.Prof.Peter Schulz

## Semester: Summer 2020

Author 1: Bajpai, Adwait
Matriculation Number: 7206731

Author 2: Ananya Reginald Frederick
Matriculation Number: 7207003

Author 3: Mahavishnu Rajaram
Matriculation Number: 7207099

# Abstract

This report is a part of VHDL homework assignment. The report details the design decision, including low level details of the audio mixer implementation. It summarizes the understanding of the group, including the output files and screen shots where appropriate.

# Contents

**Chapter 1 Introduction** ..... **1**

**Chapter 2 System Requirements** ..... **2**

2.1. Introduction ..... 2
2.2. Approach ..... 2
2.3. Multiplication Operation ..... 3
2.4. Signed and Unsigned Numbers ..... 3
2.5. Conversion Decibels to gain ..... 3

**Chapter 3 Design and Implementation** ..... **4**

3.1. Functional Block Diagram ..... 4
3.2. Design Methodology ..... 4
3.3. Testbench ..... 7
3.4. Simulation Results ..... 9

**Chapter 4 Verification and Validation** ..... **10**

4.1. Input Audio to Bitstream Conversion ..... 10
4.2. Manual Verification ..... 12

**Chapter 5 Synthesis Diagrams** ..... **14**

**Chapter 6 Conclusion** ..... **17**

**List of Figures** ..... **19**

**List of Tables** ..... **20**

**Bibliography** ..... **21**

# Chapter 1

# Introduction

This report is split into chapters depending on the approach taken to solve the problem statement. After initial discussion, the team had various approaches. Each of the approaches is described in a subsequent chapter, and one solution is finally selected, justifying the limitations / problems faces with other approaches. Overall, a V-Cycle approach is takes to solve the problem statement. We found this assignment to be thoughtfully challenging. Sufficient knowledge gained during lessons and exercises helped us tackle the problems faced.

# Chapter 2

# System Requirements

## 2.1. Introduction

The task is to design an audio mixer given a system description. The system consists of four input channels and two output channels. Input and output channels are serial TDM with 48kHz frame rate. Input TDM stream is a 16 bit, whereas output TDM stream is 24 bits. The system comprises of two gain multipliers, one of which is a master volume control. Each of the gain factors are 10bit numbers. Every target channel is a weighted sum of input channels, along with a master volume control. Number format of the input is 1.15 fixed point, output is 1.23 fixed point and gain is 5.5 fixed point. Also, number of multipliers in the output is to be limited to 2.

## 2.2. Approach

V-Cycle approach of system development has been used for this assignment. It was a good practice to approach an engineering problem. It helped structure the problem statement and use the milestones to track progress. A few iterations have been repeatedly performed, e.g., between verification and design, design and requirements, verification led us to requirement analysis and bridge the gap in our understanding of requirements.

.

Summary of the steps followed:

- Problem statement investigation and discussions
- Elaboration of deigns and implementations
- Synthesis
- Functional Verification
- Validate if the output matches the desired technical requirements.

## 2.3. Multiplication Operation

Multiplication is one of the prime signal processing operations in an audio mixer. As bits are dealt with, multiplication is binary, and can be carried out on signed and unsigned numbers. Binary multiplication is performed in a similar manner to performing decimal multiplication. The upper number is called the multiplicand and the lower number is called the multiplier. Each individual digit within the multiplier is multiplied with the entire multiplicand, starting with the least position. The result of this interim multiplication is called the partial product. The partial product is recorded with its least significant digit aligned with the corresponding position of the multiplier digit. All partial products are summed to create the final product of the multiplication. This process is often called the shift and add approach. Signed multiplication is performed using the rules of arithmetic multiplication. For negative numbers, the two's complement is taken on it to produce its equivalent magnitude, positive representation. The multiplication is then performed on the positive values. The final step is to apply the correct sign to the product. If the product should be negative due to one of the inputs being negative, the sign is applied by taking the two's complement on the result.

## 2.4. Signed and Unsigned Numbers

Negative numbers need to be represented in a real digital circuitry. In a real digital circuit, the circuits can only deal with 0's and 1's. As the hardware has only 0's and 1's, a bit is used to represent whether a number is positive or negative. This is referred to as the sign bit. If a binary number has no negative values, then it is called an unsigned number, and it represents only positive numbers. If a binary number represents negative numbers, it is called a signed number. It is important to always keep track of the type of number used as the same bit values can represent very different numbers depending on the coding mechanism being used. In a digital circuitry, two's complement is used to represent negative numbers.

## 2.5. Conversion Decibels to gain

Amplification factor is the extent to which an active device boosts the strength of a signal. Amplification is the ratio of Vout/Vin. Voltage gain in d big given by 20log(Vout/Vin). The amount of increase in audio is often expressed in db. Sound perceived as loud needs an increase in sound level by 10db.

| Decibels (Db) | Gain (Vo/Vin) | Gain in binary |
|---|---|---|
| 10 | 3.162278 | 00011.00101 |
| -5 | 0.562341 | 00000.10001 |
| 15 | 5.623413 | 00101.10011 |

**Table 2.1: Gains used in Audio Mixer**

# Chapter 3

# Design and Implementation

## 3.1. Functional Block Diagram

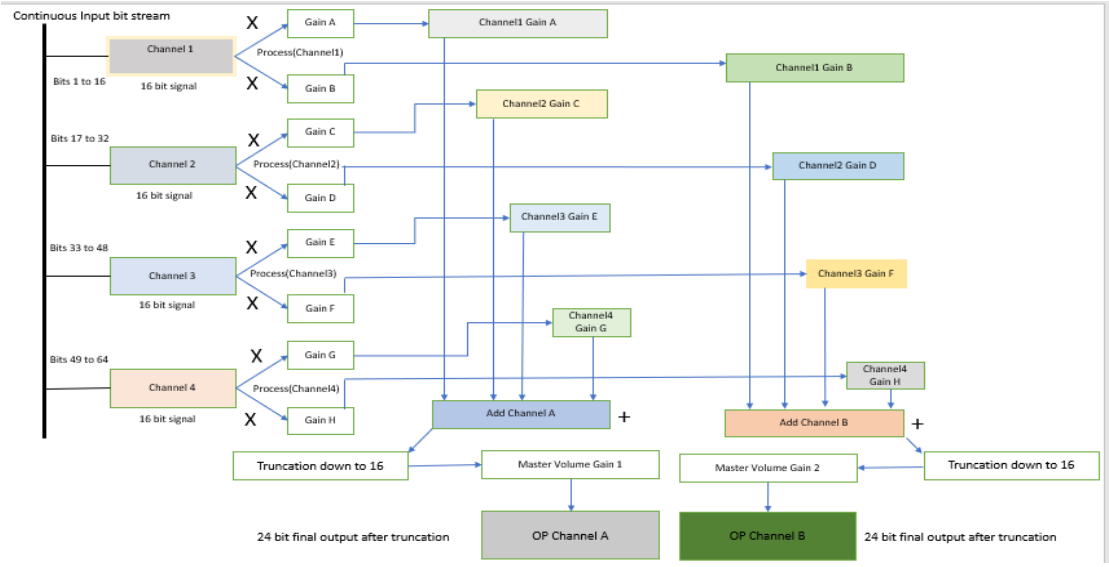



**Figure 3.1: Functional Block Diagram**

## 3.2. Design Methodology

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.ALL;
use STD.textio.all;
--##############################################################################################################
--#Entity Declaration for Audio Mixer                                                                          #
--#Port Declarations                                                                                           #
--##############################################################################################################
entity Audio_Mixer is
generic(Channel_width : natural := 15;
        gain_width : natural := 10;
        OPChannel_width : natural := 23);
        port(serialbit : IN std_logic;
        count : IN unsigned(2 downto 0);
        bitcount : in std_logic;

        --Setting the gain for each channels
        GainChannelA : IN unsigned(gain_width-1 downto 0);
        GainChannelB : IN unsigned(gain_width-1 downto 0);

        --setting master gain
        MasterVolumeGain1 : IN unsigned(gain_width-1 downto 0);
        MasterVolumeGain2 : IN unsigned(gain_width-1 downto 0);
        clk : IN std_logic;

        --Output channel
        OPChannelA: OUT signed(OPChannel_width downto 0);
        OPChannelB: OUT signed(OPChannel_width downto 0));
    end Audio_Mixer;
```

**Figure 3.2: Entity Declaration of Audio Mixer**

The Entity of the Audio Mixer, Figure 3.1, contains the input, output and clock pins. It consists of: -

- A serial input bit to take the data bit by bit from the generated binary file.
- Two gain channels.
- Two master volume gain channels
- Clock signal
- A counter
- Two output channels

```
--#############################################################################################################################
--#Architecture of Audio Mixer                                                                                                 #
--#############################################################################################################################

architecture behavioral of Audio_Mixer is

signal channel_inter : std_logic_vector(Channel_width downto 0):= (others=>'0');
signal channell: signed(Channel_width downto 0);

--Control signals
signal channel_control : std_logic := '0';
signal Calculation_control :std_logic :='0';

--Signals for first channel and gain multiplication
signal ChannellgainA : signed(26 downto 0);
signal Channel2gainA : signed(26 downto 0);
signal Channel3gainA : signed(26 downto 0);
signal Channel4gainA : signed(26 downto 0);
signal ChannellgainB : signed(26 downto 0);
signal Channel2gainB : signed(26 downto 0);
signal Channel3gainB : signed(26 downto 0);
signal Channel4gainB : signed(26 downto 0);

--signal for storing value after channel addition
signal AddChannelA : signed(28 downto 0);
signal AddChannelB : signed(28 downto 0);

--Truncating Signal
signal Inter_trunc_chA : signed(Channel_width downto 0);
signal Inter_trunc_chB : signed(Channel_width downto 0);

--Truncated storing mater volume
signal ChannelAMV : signed(26 downto 0);
signal ChannelBMV : signed(26 downto 0);

--function to multiply gain
function gainmultiply(channel : signed(15 downto 0);
                        gain: unsigned(gain_width-1 downto 0)) return signed is
                      variable ChannelGain : signed(26 downto 0) :=(others=>'0');
                      variable intermediate_gain : unsigned (10 downto 0) :=(others=>'0');
                      begin
                      intermediate_gain(9 downto 0):= gain;
                      ChannelGain:=channel*signed(intermediate_gain);
                      return ChannelGain;
                      end function;
```

**Figure 3.3: Architecture of Audio Mixer**

In Figure 3.2, the signals which will be used in the architecture are declared and also provided initial values if required. There are signals declared for storing the channels and gains multiplications, adding the channels, storing intermediate truncated signal and the signals for storing the final multiplication with the master volume. It also contains the function which will be used to multiply the channel with the respective gain and returns the value of the multiplication.

```
--#############################################################################################################################
--#Obtaining a 16 bit channel from the input file                                                                             #
--#############################################################################################################################

process(bitcount)
begin
channel_inter<= channel_inter(14 downto 0) & serialbit;
end process;

--Receiving the channels one at a time.
process(count)
begin
channell <= signed(channel_inter);
channel_control<= not channel_control;
end process;
```

**Figure 3.4: 16 bit channel from input file**

Figure 3.3 deals with taking the bit by bit input and segregating the channels. It makes use of a Serial to parallel converter. As shown in the above diagram, the bits are

slowly shifted into the 16-bit channel. The process will continue taking in and adding the values to the channel till the “count” signal is modified, in which case another process is activated which stores the channel so that the required operations can be performed on it.

```
--#################################################################################################################################
--#Multiplying the individual chaneels with respective gains based on the channel received                                       #
--#################################################################################################################################
process(channel_control)
begin
if count = "000" then
ChannellgainA<= gainmultiply(channel1,GainChannelA);
ChannellgainB<= gainmultiply(channel1,GainChannelB);
end if;
end process;

process(channel_control)
begin
if count = "001" then
Channel2gainA<= gainmultiply(channel1,GainChannelA);
Channel2gainB<= gainmultiply(channel1,GainChannelB);
end if;
end process;

process(channel_control)
begin
if count = "010" then
Channel3gainA<= gainmultiply(channel1,GainChannelA);
Channel3gainB<= gainmultiply(channel1,GainChannelB);
end if;
end process;

process(channel_control)
begin
if count = "011" then
Channel4gainA<= gainmultiply(channel1,GainChannelA);
Channel4gainB<= gainmultiply(channel1,GainChannelB);
Calculation_control <= not Calculation_control;
end if;
end process;
```

**Figure 3.5: Multiplication of individual channels with channel gain**

After the channel has been stored, the signal “count” specifies the number of the channel and the based on this the channel is Multiplied with its respective gains. The values are then stored in signals.

```
--#################################################################################################################################
--#1)Adding the product of the gain and channels to get 2 different channels                                                      #
--#2)Multiplying the new channels with Master Volume                                                                             #
--#3)Truncating extra bits                                                                                                       #
--#################################################################################################################################
process(Calculation_control)
begin

--Adding the multiplication of the channels with the gains
AddChannelA<= ("00"&ChannellgainA) + ("00"&Channel2gainA) + ("00"&Channel3gainA) + ("00"&Channel4gainA);
AddChannelB<= ("00"&ChannellgainB) + ("00"&Channel2gainB) + ("00"&Channel3gainB) + ("00"&Channel4gainB);

--Intermediate truncation
Inter_trunc_chA <= AddChannelA(28 downto 13);
Inter_trunc_chB <= AddChannelB(28 downto 13);

--with truncation
ChannelAMV<= gainmultiply(Inter_trunc_chA,MasterVolumeGain1);
ChannelBMV<= gainmultiply(Inter_trunc_chB,MasterVolumeGain2);

--output with truncation
OPChannelA<= ChannelAMV(26 downto 3);
OPChannelB<= ChannelBMV(26 downto 3);

end process;

end behavioral;
```

**Figure 3.6: Multiplication of Master Gain Control, Truncation and Summation**

This is the final part of the code. Once the 4 channels have been segregated and multiplied with their respective gains, the product of the channels with gain is added and stored in 2 different channels. The addition results form a signal of 29 bits and its multiplication with a 10-bit master volume (11 bit if we convert it to signed) will result in a 40-bit signal. Performing such a huge multiplication will result in the use of 2 multipliers (MULT18X18B). To reduce the number of multipliers to be used, truncation was performed. The 29-bit signal was truncated to 15 bits keeping the 15 most significant bits. By performing this truncation, the multipliers used in the multiplication of the channels with their respective gains were reused thereby saving the use of more multipliers.

“ChannelAMV” and “ChannelBMV” were used to save the multiplication of the truncated signal with the master volume. After this multiplication we are left with 27 bits, but the final channels have to be of 24 bits each and therefore 3 more least significant bits were truncated from “ChannelAMV” and “ChannelBMV”. Values of both were stored in the output signals “OPChannelA” and “OPChannelB”.

## 3.3. Testbench

```
--#############################################################################################################################
--#Architecture of Audio Mixer Testbench                                                                                      #
--#############################################################################################################################
architecture Behavioral of Audio_Mixer_TB is

component Audio_Mixer is
generic(Channel_width : natural := 15;
        gain_width : natural := 10;
        OPChannel_width : natural := 23);
port(serialbit : IN std_logic;
     count : IN unsigned(2 downto 0);
     bitcount : in std_logic;
     --Setting the gain for each ;
     --Setting the gain for each channels
     GainChannelA : IN unsigned(gain_width-1 downto 0);
     GainChannelB : IN unsigned(gain_width-1 downto 0);
     --setting master gain
     MasterVolumeGain1 : IN unsigned(gain_width-1 downto 0);
     MasterVolumeGain2 : IN unsigned(gain_width-1 downto 0);
     clk : IN std_logic;
     OPChannelA: OUT signed(OPChannel_width downto 0);
     OPChannelB: OUT signed(OPChannel_width downto 0));
end component;

--Serial entry for channels
signal serialbit : std_logic:='0';
--Count
signal count : unsigned(2 downto 0):=(others=>'1');
signal bitcount : std_logic:='0';
--Signal InputFrame : signed(Channel_width-1 downto 0):=(others=>'0');
signal GainChannelA : unsigned(gain_width-1 downto 0):=(others=>'0');
signal GainChannelB : unsigned(gain_width-1 downto 0):=(others=>'0');
--Setting master gain
signal MasterVolumeGain1 : unsigned(gain_width-1 downto 0):=(others=>'0');
signal MasterVolumeGain2 : unsigned(gain_width-1 downto 0):=(others=>'0');
signal clk : std_logic:='0';
signal OPChannelA: signed(OPChannel_width downto 0);
signal OPChannelB: signed(OPChannel_width downto 0);

--Needed constant values
constant clock_period: time := 20833.33 ns;
```

**Figure 3.7: Architecture of Audio Mixer Testbench**

The architecture of the Audio Mixer testbench starts here. This part includes the declaration of the signals and the components that the architecture is going to use. Also, the signals that need to be used in the portmap are declared here. The will later be port mapped with the ports of the component being used by the architecture.

```
begin
--#############################################################################################################################
--#Device Under Test                                                                                                          #
--#############################################################################################################################
dut: Audio_Mixer
generic map( Channel_width=>Channel_width,
                        gain_width=>gain_width,
             OPChannel_width=>OPChannel_width)
port map( serialbit=>serialbit,
          count=>count,
          bitcount=>bitcount,
          GainChannelA=>GainChannelA,
          GainChannelB=>GainChannelB,
          MasterVolumeGain1=>MasterVolumeGain1,
          MasterVolumeGain2=>MasterVolumeGain2,
          clk=>clk,
          OPChannelA=>OPChannelA,
          OPChannelB=>OPChannelB);

process
begin
clk <= '1';
wait for clock_period/2;
clk<= '0';
wait for clock_period/2;
end process;
```

**Figure 3.8: Design under Test**

DUT is used to declare the input and output ports of the DUT. It instantiates the DUT and connects the ports of the DUT to the declared signals as it can clearly be seen in the above Figure. Along with this it also instantiates the processes that drive and exchange values with the signals that are connected with the input ports of the DUT. In the above case it maps the ports with the signals that were declared at the beginning of the Architecture

```
--#############################################################################################################################
--#1)Defining the file pointer for Read and Write                                                                             #
--#2)Reading data from the file bit by bit in single TDM stream                                                              #
--#3)Writing output data into file.                                                                                          #
--#############################################################################################################################
stim: process
file file_pointer: text;
file file_pointer_write: text;
variable line_content: std_logic;
variable line_content_write: std_logic_vector(23 downto 0);
variable Line_number: line;
variable Line_number_write: line;
variable j: integer:=0;
variable part_one:std_logic;
variable combined:std_logic_vector(63 downto 0);
begin
file_open(file_pointer,"Newaudio.txt",READ_MODE);
file_open(file_pointer_write,"OutputFile.txt",Write_mode);
while not endfile(file_pointer) loop
readline (file_pointer,Line_number);
read(Line_number,line_content);
part_one:= line_content;
serialbit<= part_one;
bitcount<= not bitcount;
if j=16 then
count<="000";
GainChannelA<="0011000000";
GainChannelB<="0011000000";

elsif j=32 then
GainChannelA<="0011000000";
GainChannelB<="0111000000";
count<="001";

elsif j=48 then
count<="010";
GainChannelA<="0011000000";
GainChannelB<="0011000000";

elsif j=64 then
count<="011";
GainChannelA<="0011000000";
GainChannelB<="0011000000";
MasterVolumeGain1<="0011000000";
MasterVolumeGain2<="0011000000";
j:=0;
line_content_write:=std_logic_vector(OPChannelA);
```

**Figure 3.9: File input and output operations**

The testbench feed values to the main VHDL code and gets the output. This plays a very major role in the verification of the main code. The above figure shows the process which is feeding values in the form of input bits, gains, and count. This code is currently reading the binary file generated from the tone created in Octave/Matlab. It runs a while loop which reads each bit from the file and then informs the main code every 16 times. This information which is passed via the signal “count” lets the VHDL code know that channel is received and gives it the respective gain values based on which channel it has received. It also keeps track of which channel has been received and passes this information to the main code. It also handles the data write into the output file (OutputFile.txt).

## 3.4. Simulation Results

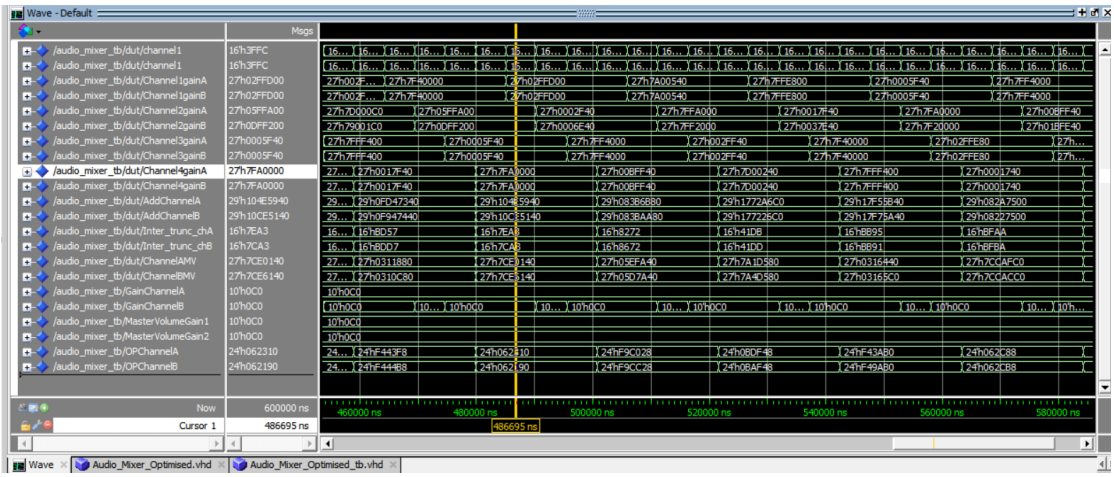



**Figure 3.10: Simulation Results Part 1**

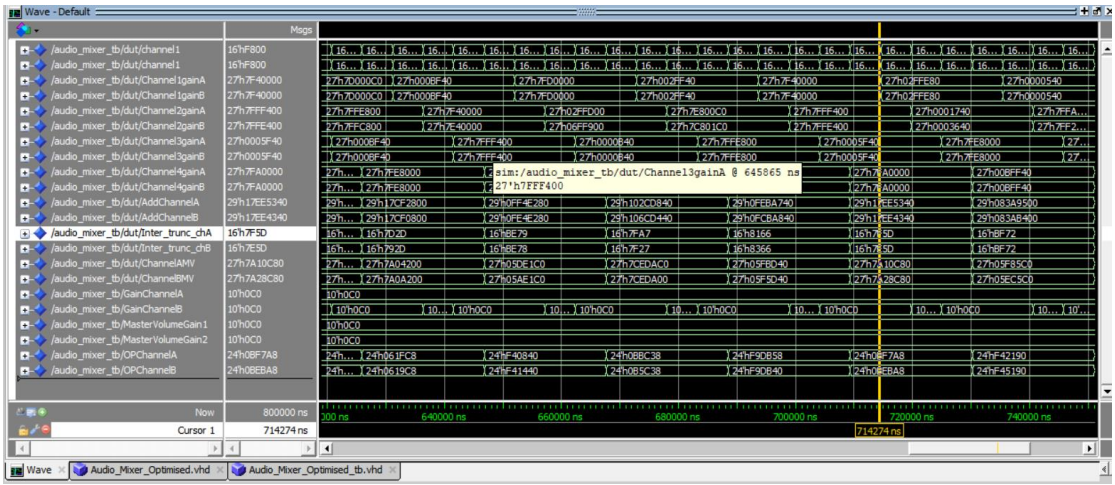



**Figure 3.11: Simulation Results Part 2**

# Chapter 4

# Verification and Validation

## 4.1. Input Audio to Bitstream Conversion

The below give audio files are converted from 44.1KHz to 48KHz frequency with 16bits/frame. The audio input files are as shown below

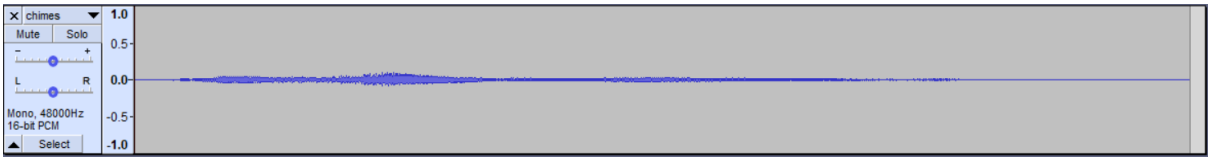



**Figure 4.1: Audio File input 1**

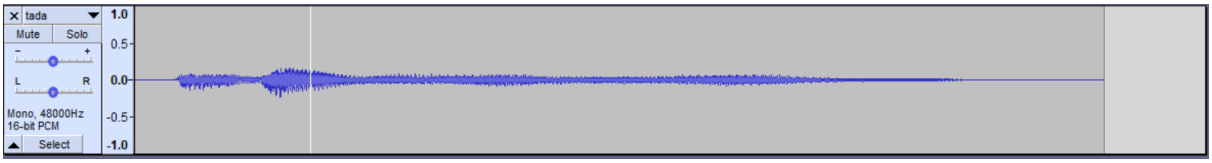



**Figure 4.2: Audio File input 2**

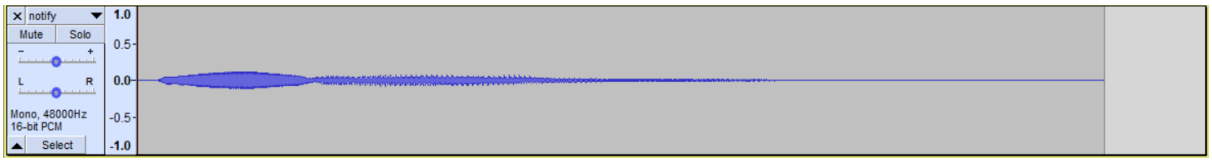



**Figure 4.3: Audio File input 3**

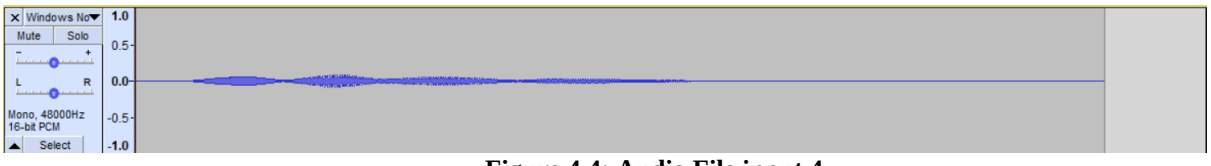



**Figure 4.4: Audio File input 4**

These audio files are converted to a bit stream with 16bit of each audio followed by the other forming frames. We have taken 1sec audio from this, so all these 4 audio data combined we have 48000 bits (Since frequency is 48KHz, it will be processed at 48000bps). Conversion is done in Matlab using the

below code. Pre-processing is used to convert a .wav to .txt file, which is then given as an input to DUT and used for verification.

Final_AudioMixer.m × Op2Audio.m ×

```
int16 binn1;
int16 binn2;
int16 binn3;
int16 binn;
%SamplingFrequenciess
Fs=48000;
Fs1=48000;
Fs2=48000;
Fs3=48000;
%NoOfSamples
samples=[1,Fs];
samples1=[1,Fs1];
samples2=[1,Fs2];
samples3=[1,Fs3];
%Read 4 different audio files for 4 different inputchannel(16bit)
[binn,Fs]=audioread("chimes.wav",samples,"native");
[binn1,Fs1]=audioread("tada.wav",samples1,"native");
[binn2,Fs2]=audioread("notify.wav",samples2,"native");
[binn3,Fs3]=audioread("Windows Notify.wav",samples3,"native");

info=audioinfo('chimes.wav');
info1=audioinfo('tada.wav');
info2=audioinfo('notify.wav');
info3=audioinfo('Windows Notify.wav');
% File to output the bitstream at the rate of 48KHz of merged audio(4
% audios) information
FID1=fopen("Outputaudio.txt",'w+');
%File error handling
if FID1 < 0
msgbox(['Cannot read file: ', "audio.bin"],'File Error','Error');
```

**Figure 4.5: Pre-Processing part1 in MATLAB for .wav file**

Final_AudioMixer.m × Op2Audio.m ×

```
end
%The 4 audio's decimal is converted to  binary format
Data= dec2bin(binn);
Data3= dec2bin(binn1);
Data4= dec2bin(binn2);
Data5= dec2bin(binn3);
a=1;
b=1;
c=1;
d=1;
%while loop that 12000 times, each time 4 different audio file's 16bit
% information is written consecutively to form a total of 48000bits )
while a < 750*16
for k= a:(16+(a-1))
FinalData=Data(a:k);
fprintf(FID1,FinalData);
fprintf(FID1,'\n');
a=a+1;
end
for l=b:(16+(b-1))
FinalData=Data3(b:l);
fprintf(FID1,FinalData);
fprintf(FID1,'\n');
b=b+1;
end
for m=c:(16+(c-1))
FinalData=Data4(c:m);
fprintf(FID1,FinalData);
fprintf(FID1,'\n');
c=c+1;
```

**Figure 4.6: Pre-Processing part2 in MATLAB for .wav file**

```
61 -    end
62 -    for n=d:(16+(d-1))
63 -    FinalData=Data5(d:n);
64 -    fprintf(FID1,FinalData);
65 -    fprintf(FID1,'\n');
66 -    d=d+1;
67 -    end
68 -    end
69 -    fclose(FID1);
```

**Figure 4.7: Pre-Processing part3 in MATLAB for .wav file**

Then the output bitstream is processed using VHDL and the output bit stream is then again processed Matlab and converted to an output audio file.

Final_AudioMixer.m | Op2Audio.m

```
1 -  Fs=48000;
2 -  FID=fopen("OutputFile.txt",'rb');
3 -  if FID < 0
4 -  msgbox(['Cannot read file: ', "24bitAudioOp"],'File Error','Error');
5 -  end
6 -  FID1=fopen("RemoveNewLine1",'a+');
7 -  if FID1 < 0
8 -  msgbox(['Cannot read file: ', "RemoveNewLine"],'File Error','Error');
9 -  end
10   %removing 'new line'
11 -  Data=fgetl(FID);
12 -  fprintf(FID1,Data);
13 -  i=1;
14 -  while ischar(Data)
15 -  Data=fgetl(FID);
16 -  fwrite(FID1,Data);
17 -  i=i+1;
18 -  end
19   %scanning the 24bit information as signed integer
20 -  B=fscanf(FID1,'%24d',[1,inf]);
21   %writing the data in audio format
22 -  audiowrite("Audio_Output.wav",B,Fs);
```

**Figure 4.8: Post Processing for conversion to .wav output file**

## 4.2. Manual Verification

Apart from verification via sampling, Manual Verification was also carried out. The steps followed in the calculation are as follows:

- Each input channel was multiplied by 2 gains e.g. Gain A and Gain B and their values were stored.
- Once all the gain multiplication of the 4 channels are available the values are added. All values multiplied by Gain A are added together and same is done with the channel values multiplied by Gain B. They shall be named “AddChannelA” and “AddChannelB” respectively.
- Now, each of these channels should be multiplied with a Master volume. But to reduce the use of Multipliers each of the channel is truncated first to 16 bits each.
- These 16-bit channels are now multiplied by the “MasterVolumeGain1” and MasterVolumeGain2” respectively and stored in “MultipyA” and “MultiplyB”
- Both “MultipyA” and “MultiplyB” are truncated to two 24-bit data channels.

As shown in Figure 4.9 and Figure 4.10 the output in both the cases is **completely equal**. Hence we know that the code is verified and following the algorithm.

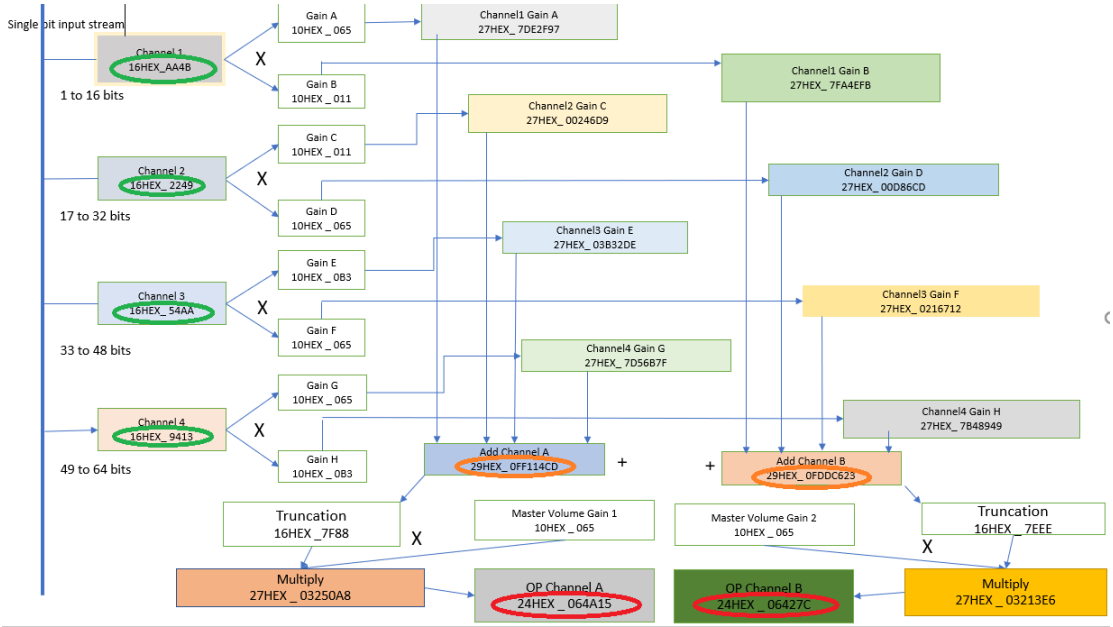



**Figure 4.9: Values obtained from manual calculation**

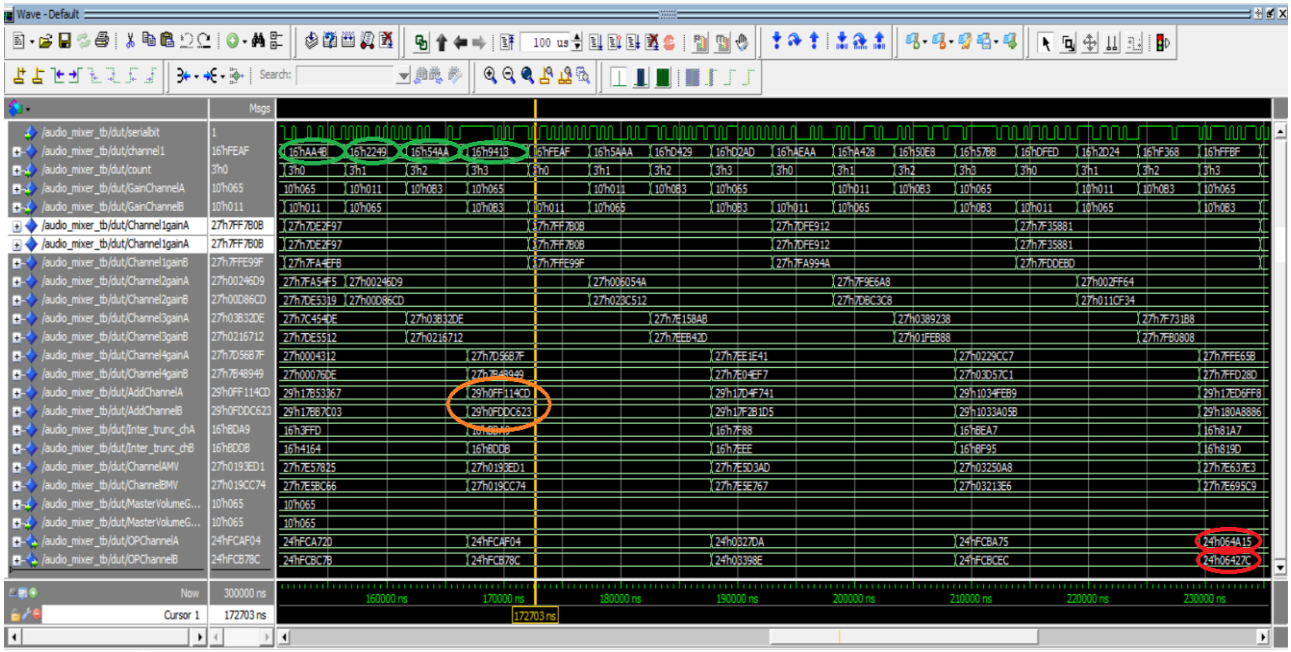



**Figure 4.10: Values found on the Simulation Window**

# Chapter 5

# Synthesis Diagrams

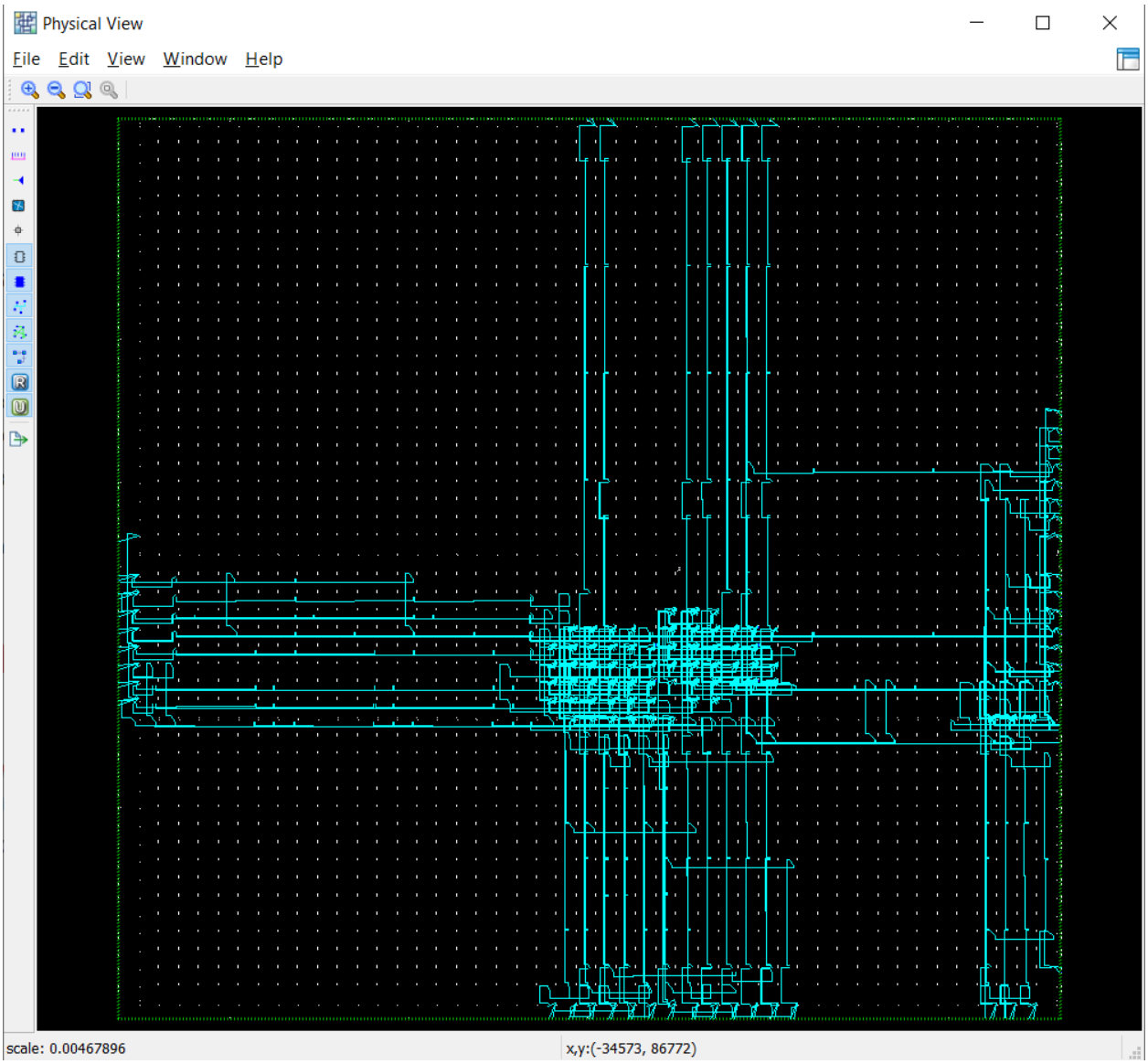



**Figure 5.1: Physical View**

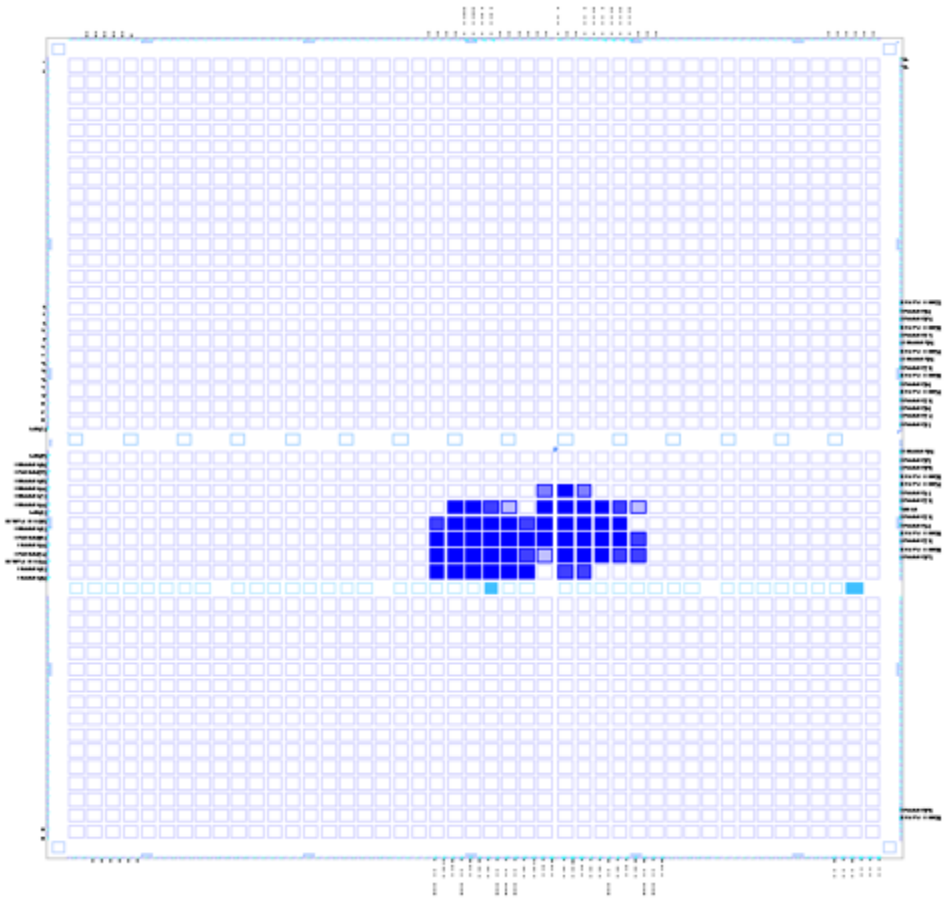


**Figure 5.2: Floor Plan View**

```
DSP primitives:       2 of 15 (13%)

Details:
CCU2B:          104
GSR:            1
IB:             44
MULT18X18B:     2
OB:             48
ORCALUT4:       100
PUR:            1
VHI:            1
VLO:            1
Mapper successful!
```

**Figure 5.3: Justifying the number of multipliers i.e. 2**

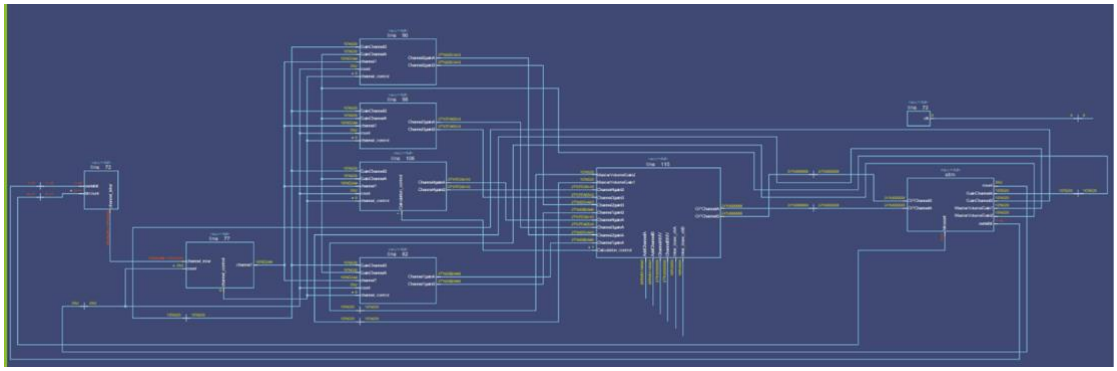


**Figure 5.4: Dataflow Model**

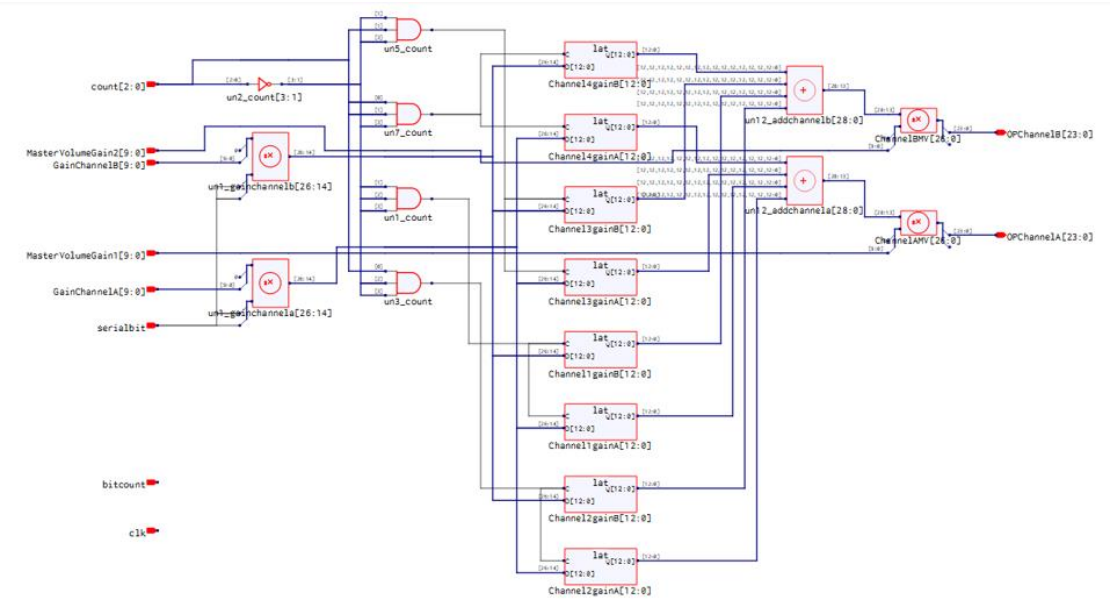



**Figure 5.5: RTL View**

Register transfer level modelling is a level of design abstraction. Vector data is moved and operated on in a synchronous manner. RTL design methodology is widely used in data path modelling and computer system design. A register is a group of D-Flip-Flops running on the same clock, reset, and enable inputs. Data is moved in and out of the bank of D-flip-flops as a vector. Logic operations can be made on the vectors and are latched into the register on a clock edge. A register is a higher level of abstraction that allows vector data to be stored without getting into the details of the lower-level implementation of the D flip-flops and combinational logic.

| Vhdl_Final project summary | | | |
|---|---|---|---|
| Module Name: | Vhdl_Final | Synthesis: | SynplifyPro |
| Implementation Name: | impl1 | Strategy Name: | Strategy1 |
| Last Process: | VHDL Simulation File | State: | Passed |
| Target Device: | LFXP2-17E-5QN208C | Device Family: | LatticeXP2 |
| Device Type: | LFXP2-17E | Package Type: | PQFP208 |
| Performance grade: | 5 | Operating conditions: | COM |
| Logic preference file: | Vhdl_Final.lpf | | |
| Physical Preference file: | impl1/Vhdl_Final_impl1.prf | | |
| Product Version: | 3.11.0.396.4 | Patch Version: | |
| Updated: | 2020/11/01 03:18:28 | | |
| Implementation Location: | D:/Softwares/lscc/diamond/3.11_x64/bin/nt64/impl1 | | |
| Project File: | D:/Softwares/lscc/diamond/3.11_x64/bin/nt64/Vhdl_Final.ldf | | |

**Figure 5.6: Project Summary**

# Chapter 6

# Conclusion

Learning and Results

In this project an Audio Mixer was Designed and synthesized in FPGA technology Lattice XP2 and device XP2-17 PQFP 208. It was a great learning experience. New tools were learnt and difficulties were overcome.

Major Challenges Faced and concepts learnt:

Understanding and visualizing how this particular device worked.

- Was dealt with by reading up on how Audio Mixers worked to understand what was expected.

Dealing with Data Saturation, Conversion, Overflow and Truncation:

- Data Saturation:- There is a fixed range between maximum and minimum values and as such additional and multiplications fixed. Subtracting a number to less than it minimum range will not give us the desired result.
- Conversion:- Converting a number from signed to unsigned is tricky as the most significant bit may be '1' which would make it a negative number when converted to signed when it is actually not a negative number. Therefore, when dealing with unsigned to signed conversion the unsigned data should always be padded with a '0'.
- Overflow: Adding multiple operands results in the possibility of there being a carry over value which will not be accommodated if it exceeds the range of the data range. So to deal with it we should figure out how much the carry over will be and store it in a bigger variable/signal and also we should append the required bits in front of the operands e.g. add<= ("00" &a) + ("00" &b) + ("00" &c) + ("00" &d)
- Truncation: The bits which are undesired can be removed (preferably from the least significant bit). E.g. a(2 downto 0) = b(4 downto 2)

Reducing the number of multipliers and optimizing code:

- Try to reuse the multiplication operator as much as possible rather than creat multiple instances of the code.
- Make one set of operands use the operator at a time.

Conversion of .wav file to binary and vice versa.
Understanding the concept of TDM
Using text files to provide input and store output.

**Tools used:**

- Anaconda and Matlab for .wav to binary conversion and vice versa.
- ModelSim for Design and Testbench implementation
- Lattice Diamond for Design Synthesis
- Synplify for RTL View generation

# List of Figures

Figure 3.1: Functional Block Diagram ..... 4
Figure 3.2: Entity Declaration of Audio Mixer ..... 4
Figure 3.3: Architecture of Audio Mixer ..... 5
Figure 3.4: 16 bit channel from input file ..... 5
Figure 3.5: Multiplication of individual channels with channel gain ..... 6
Figure 3.6: Multiplication of Master Gain Control, Truncation and Summation ..... 6
Figure 3.7: Architecture of Audio Mixer Testbench ..... 7
Figure 3.8: Design under Test ..... 8
Figure 3.9: File input and output operations ..... 8
Figure 3.10: Simulation Results Part 1 ..... 9
Figure 3.11: Simulation Results Part 2 ..... 9
Figure 4.1: Audio File input 1 ..... 10
Figure 4.2: Audio File input 2 ..... 10
Figure 4.3: Audio File input 3 ..... 10
Figure 4.4: Audio File input 4 ..... 10
Figure 4.5: Pre-Processing part1 in MATLAB for .wav file ..... 11
Figure 4.6: Pre-Processing part2 in MATLAB for .wav file ..... 11
Figure 4.7: Pre-Processing part3 in MATLAB for .wav file ..... 12
Figure 4.8: Post Processing for conversion to .wav output file ..... 12
Figure 4.9: Values obtained from manual calculation ..... 13
Figure 4.10: Values found on the Simulation Window ..... 13
Figure 5.1: Physical View ..... 14
Figure 5.2: Floor Plan View ..... 15
Figure 5.3: Justifying the number of multipliers i.e. 2 ..... 15
Figure 5.4: Dataflow Model ..... 15
Figure 5.5: RTL View ..... 16
Figure 5.6: Project Summary ..... 16

# List of Tables

Table 2.1: Gains used in Audio Mixer.....3

# Bibliography

[1] http://www.sengpielaudio.com/calculator-gainloss.htm

[2] https://fpgaer.wordpress.com/parallel-to-serial-converter/

[3] https://thehackerdiary.wordpress.com/2017/06/09/it-is-ridiculously-easy-to-generate-any-audio-signal-using-python/

[4] https://fpgasite.wordpress.com/serial-to-parallel-converter/

[5] https://www.exploringbinary.com/twos-complement-converter/

[6] https://riptutorial.com/vhdl